



# Tree Methods

Let's learn something!



# Python and Spark

- A very powerful group of algorithms falls under the “Tree Methods” title.
- We’ll discuss decision trees, random forests, and gradient boosted trees!



# Reading Assignment

Chapter 8 of  
**Introduction to Statistical Learning**  
By Gareth James, et al.



## Tree Methods

- Let's start off with a thought experiment to give some motivation behind using a decision tree method.



## Tree Methods

- Imagine that I play Tennis every Saturday and I always invite a friend to come with me.
- Sometimes my friend shows up, sometimes not.
- For him it depends on a variety of factors, such as: weather, temperature, humidity, wind etc..
- I start keeping track of these features and whether or not he showed up to play with me.



# Tree Methods

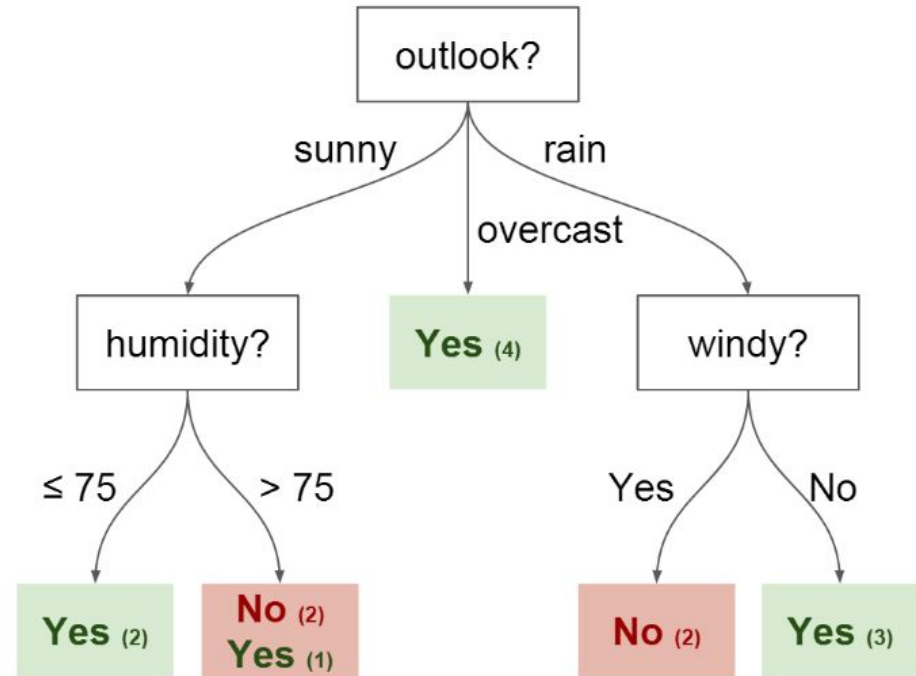
| Temperature | Outlook  | Humidity | Windy | Played?   |
|-------------|----------|----------|-------|-----------|
| Mild        | Sunny    | 80       | No    | Yes       |
| Hot         | Sunny    | 75       | Yes   | <b>No</b> |
| Hot         | Overcast | 77       | No    | Yes       |
| Cool        | Rain     | 70       | No    | Yes       |
| Cool        | Overcast | 72       | Yes   | Yes       |
| Mild        | Sunny    | 77       | No    | <b>No</b> |
| Cool        | Sunny    | 70       | No    | Yes       |
| Mild        | Rain     | 69       | No    | Yes       |
| Mild        | Sunny    | 65       | Yes   | Yes       |
| Mild        | Overcast | 77       | Yes   | Yes       |
| Hot         | Overcast | 74       | No    | Yes       |
| Mild        | Rain     | 77       | Yes   | <b>No</b> |
| Cool        | Rain     | 73       | Yes   | <b>No</b> |
| Mild        | Rain     | 78       | No    | Yes       |



# Tree Methods

I want to use this data to predict whether or not he will show up to play.

An intuitive way to do this is through a Decision Tree

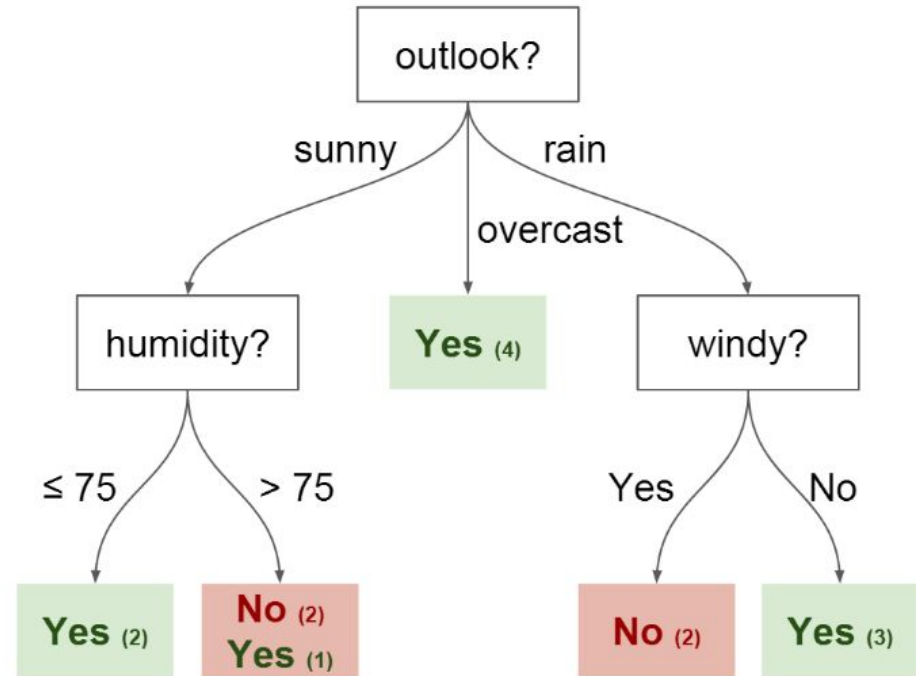




# Tree Methods

In this tree we have:

- Nodes
  - Split for the value of a certain attribute
- Edges
  - Outcome of a split to next node



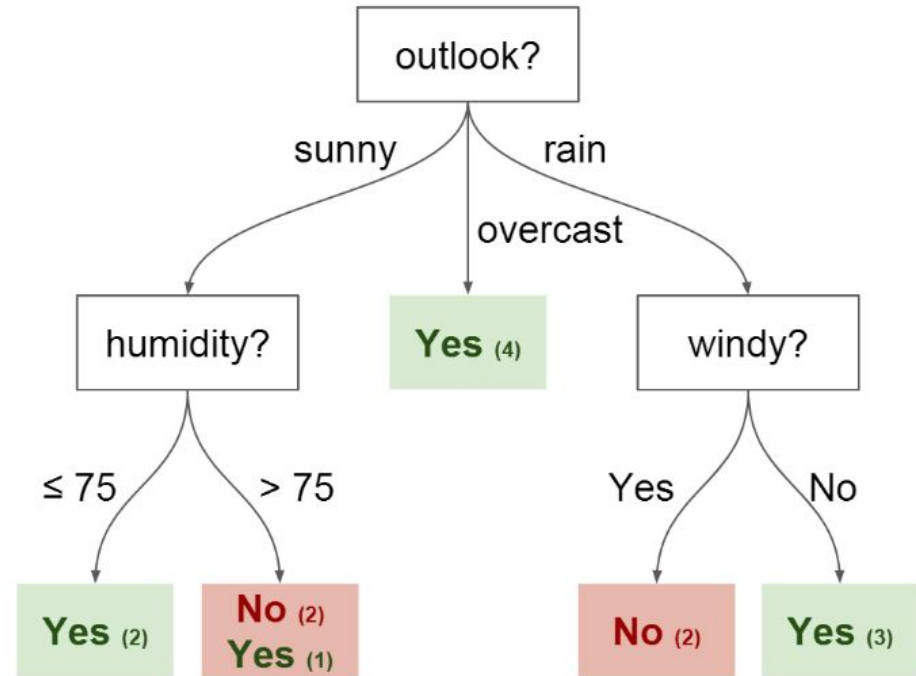




# Tree Methods

In this tree we have:

- Root
  - The node that performs the first split
- Leaves
  - Terminal nodes that predict the outcome





# Intuition Behind Splits

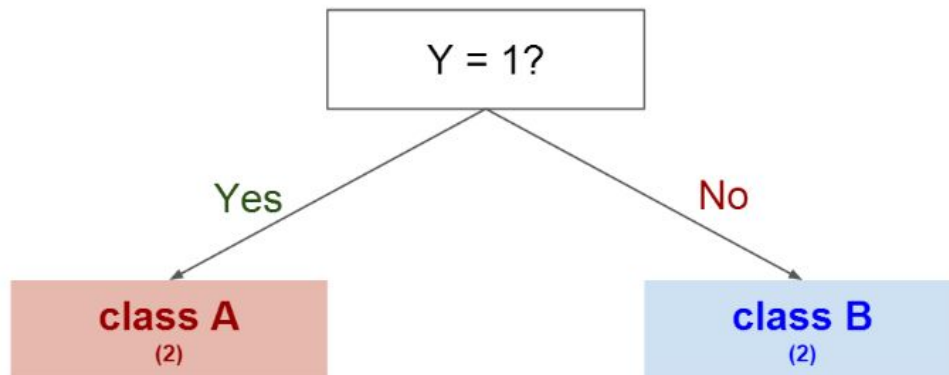
Imaginary Data with 3 features (X,Y, and Z) with two possible classes.

| X | Y | Z | Class |
|---|---|---|-------|
| 1 | 1 | 1 | A     |
| 1 | 1 | 0 | A     |
| 0 | 0 | 1 | B     |
| 1 | 0 | 0 | B     |



# Intuition Behind Splits

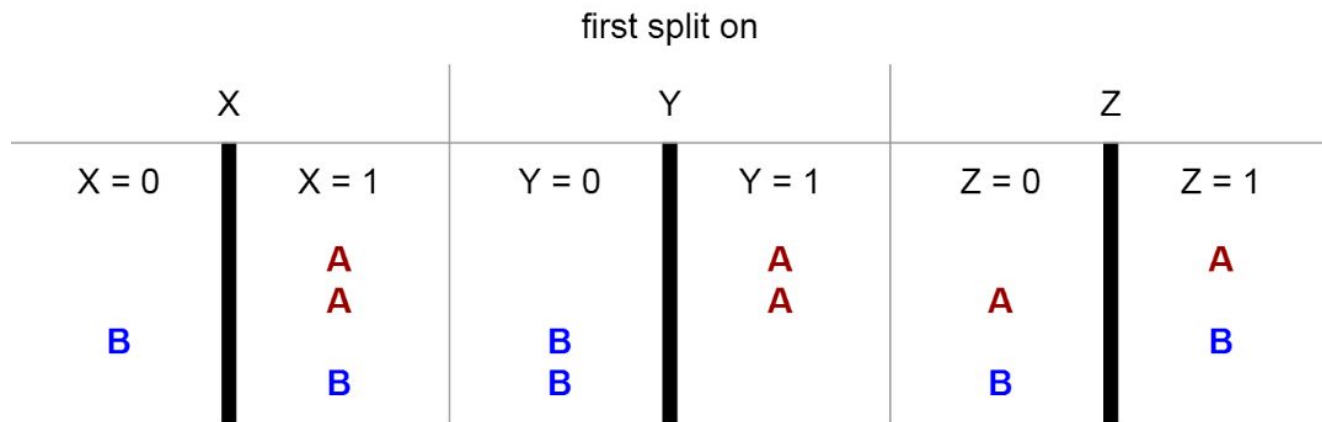
Splitting on  $Y$  gives us a clear separation between classes





# Intuition Behind Splits

We could have also tried splitting on other features first:





# Intuition Behind Splits

Entropy and Information Gain are the Mathematical Methods of choosing the best split. Refer to reading assignment.

*Entropy:*

$$H(S) = - \sum_i p_i(S) \log_2 p_i(S)$$

*Information Gain:*

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{S} H(S_v)$$



# Random Forests

To improve performance, we can use many trees with a random sample of features chosen as the split.

- A new random sample of features is chosen for **every single tree at every single split.**
- Works for both classification and regression tasks!



# Random Forests

What's the point?

- Suppose there is **one very strong feature** in the data set. Most of the trees will use that feature as the top split, resulting in an ensemble of similar trees that are **highly correlated**.



# Random Forests

What's the point?

- Averaging highly correlated quantities does not significantly reduce variance.
- By randomly leaving out candidate features from each split, **Random Forests "decorrelates" the trees**, such that the averaging process can reduce the variance of the resulting model.





# Python and Spark

- Finally, let's discuss Gradient Boosted Trees!



# Python and Spark

- Gradient boosting involves three elements:
  - A loss function to be optimized.
  - A weak learner to make predictions.
  - An additive model to add weak learners to minimize the loss function.



# Python and Spark

- Loss Function
  - A loss function in basic terms is the function/equation you will use to determine how “far off” your predictions are.



# Python and Spark

- Loss Function
  - For example, regression may use a squared error and classification may use logarithmic loss.
  - We won't have to deal with this directly using Spark.



# Python and Spark

- Weak Learner
  - Decision trees are used as the weak learner in gradient boosting.
  - It is common to constrain the weak learners: such as a maximum number of layers, nodes, splits or leaf nodes.



# Python and Spark

- Additive Model
  - Trees are added one at a time, and existing trees in the model are not changed.
  - A gradient descent procedure is used to minimize the loss when adding trees.



# Python and Spark

- So what is the most intuitive way to think about all this if Spark does this all for us?
- Here is a nice way to think about it in 3 “easy” steps



# Python and Spark

1. Train a weak model **m** using data samples drawn according to some weight distribution





# Python and Spark

2. Increase the weight of samples that are misclassified by model  $m$ , and decrease the weight of samples that are classified correctly by model  $m$



## Python and Spark

3. Train next weak model using samples drawn according to the updated weight distribution.



# Python and Spark

- In this way, the algorithm always trains models using data samples that are "difficult" to learn in previous rounds, which results an ensemble of models that are good at learning different "parts" of training data.



## Python and Spark

- Basically “boosting” weights of samples that were difficult to get correct.



## Python and Spark

- The real details of gradient boosting lies in the mathematics, which you may or may not be ready for depending on your background.
- The full details are at the end of Chapter 8 of ISLR!



## Python and Spark

- Spark handles all of this “under the hood” for you, so you can use the defaults if you want, or dive into ISLR and begin to play around with the parameters!
- Let's continue!



# Tree Methods Documentation Example

Let's learn something!



# Python and Spark

- Now that we have some intuitive understanding of how these algorithms work, let's dive into the documentation examples!





# Python and Spark

- We'll work through Decision Tree, Random Forest and Gradient Boosted Trees.
- We will also expand a little more from the documentation example and show some useful evaluation features!



# Tree Methods Code Along



# Python and Spark

- We'll work through all 3 Tree Methods discussed and compare their results on a college dataset.
- The data set has features of universities and labeled either Private or Public.
- Let's get started!



# Tree Methods Consulting Project

Let's learn something!



# Python and Spark

- You've been contracted by the Purina Dog Food company and flown out to their HQ in St. Louis, Missouri!





# Python and Spark

- You've been hired by a dog food company to try to predict why some batches of their dog food are spoiling much quicker than intended!





## Python and Spark

- Unfortunately this Dog Food company hasn't upgraded to the latest machinery, meaning that the amounts of the five preservative chemicals they are using can vary a lot, but which is the chemical that has the strongest effect?



# Python and Spark

- The dog food company first mixes up a batch of preservative that contains 4 different preservative chemicals (A,B,C,D) and then is completed with a "filler" chemical.





## Python and Spark

- The food scientists believe one of the A,B,C, or D preservatives is causing the problem, but need your help to figure out which one!



# Python and Spark

- Use Machine Learning with RF to find out which parameter had the most predictive power, thus finding out which chemical causes the early spoiling!
- So create a model and then find out how you can decide which chemical is the problem!



# Python and Spark

- Their data looks like this:
  - Pres\_A : Percentage of preservative A in the mix
  - Pres\_B : Percentage of preservative B in the mix
  - Pres\_C : Percentage of preservative C in the mix
  - Pres\_D : Percentage of preservative D in the mix
  - Spoiled: Label indicating whether or not the dog food batch was spoiled.



# Python and Spark

- Think carefully about what this problem is really asking you to solve.
- While we will use Machine Learning to solve this, it won't be with your typical train/test split workflow. If this confuses you, skip ahead to the solution code along!



# **Tree Methods Consulting Project Solutions**



# Python and Spark

- Our main task was to figure out which preservative chemical (A,B,C,D) was having an effect on the dog food being spoiled.
- So how can we use machine learning models to solve this?



# Python and Spark

- Many machine learning models produce some sort of coefficient value for each feature involved, indicating their “importance” or predictive power.
- Remember back to the documentation lecture for this section of the course!



## Python and Spark

- We mentioned that these tree method classifiers had a **.featureImportances** attribute available!
- So we can create a model, fit it on all the data, and then check which feature (preservative) was causing the spoilage!





# Python and Spark

- **.featureImportances** returns:
- `SparseVector(4, {0: 0.0026, 1: 0.0089, 2: 0.9686, 3: 0.0199})`
- Corresponding to a **features** column:
- `Row(features=DenseVector([4.0, 2.0, 12.0, 3.0]), Spoiled=1.0)`



# Python and Spark

- There are many different ways to solve this problem, including just using “pure” statistics instead of a machine learning model.



# Python and Spark

- Hopefully this consulting project shows how we can apply machine learning in a different way from previous examples.
- In this case we don't really care about test/train splits or deployments.



# Python and Spark

- What we really want to understand is the fundamental relationship between each feature column and the label itself.